

Einführung in die Programmierung

Prof. Dr. Gemma Roig
Prof. Dr. Matthias Kaschube
Dr. Karsten Tolle



Übungsblatt ÜE-07

Ausgabe: 03.12.2025
Abgabe: 20.12.2025
12:00 Uhr

Curiosity Path – Team-Blatt (2 Wochen)

Hinweise:

- Dieses Aufgabenblatt ist in **Zweierteams** zu bearbeiten. Teamarbeit ist Teil des Lerninhaltes. Die Teams sollten aus einem Tutorium stammen. Die Tutoren und Tutorinnen sind angehalten in den Tutorien die Teambildung zu unterstützen. Die finale Bewertung durch die Tutoren und Tutorinnen erfolgt dabei erst nach einem erfolgreichem Code Review. Dort wird geprüft, ob von allen Teammitgliedern der abgegebene Inhalt auch verstanden wurde. ... es geht also nicht darum die Arbeit einfach aufzuteilen! **Jeder muss alles verstehen!**

Wer keinen Partner findet, muss dies spätestens bis zum 13.12. an seine Tutorin/Tutor melden und wird dann zugeteilt. Einzelabgaben sind nur in Notfällen (Teampartner bricht weg) möglich. In diesem Fall ist unverzüglich der Tutor oder die Tutorin zu informieren. Ansonsten werden Einzelabgaben mit Null (0) Punkten bewertet.

- **Pro Team nur eine Abgabe in Moodle!**
- *Sie dürfen für dieses Blatt alles nutzen, jedoch ist es nicht zielführend automatisch generierte Lösungen (z.B. über ein LLM) einfach zu kopieren und hochzuladen. **Quellen sind anzugeben, also auch ob ein LLM bei der Erstellung unterstützt hat!***
- *Dieses Blatt ist in 2er Teams zu lösen! Mit der `__author__` Angabe (siehe nächster Punkt) bestätigen Sie, dass die Abgabe Ihr Werk ist. Die Abgaben werden auch auf **Plagiate** geprüft. Sollten wir identische Abgaben finden, erhält keiner der abgebenden Punkte.*
- *Achten Sie darauf die Variable `__author__` in allen Quellcode Dateien (.py) korrekt zu setzen (am Anfang des Quellcodes – Teamabgabe!):
`__author__ = "<Matr-Nr_1>, <Nachname_1>, <Matr-Nr_2>, <Nachname_2>"`
Beispiel: `__author__ = "1234568, Kaschube 1234567, Tolle"`
... Leerstellen vor und nach dem „`=`“ und Leerstelle nach dem Komma beachten, sowie keine spitzen Klammern verwenden.*
- *Digitale Abgaben, die nicht im Format **.pdf** oder **.txt** für Texte oder **.py** für Code erfolgen, werden nicht überprüft. Bei Abgaben mehrerer Dateien müssen diese als **.zip** zusammengefasst werden.*
- *Sollten Aufgabenstellungen nicht eindeutig sein, bitte durch Ihre eigenen Annahmen ergänzen und diese dokumentieren (z.B. in einem Kommentar).*

- Achten Sie auch auf Vorgaben Ihrer Tutorin/Tutors. Hierfür auch in das Gruppenforum im Moodle-Kurs von EPR schauen!
- Beim Programmieren und Kommentieren halten Sie sich bitte an die Regeln im Programmierhandbuch (Style Guide), siehe Moodle-Kurs. Im Zweifelsfall gilt PEP 8. ... möglichst selbst mit `pycodestyle` mal prüfen. Nicht einhalten der Konventionen führt auch zu Punktabzug.
- **Doc-Strings** sind für jedes Modul, Funktion, Klasse, Methode, ... zwingend anzugeben. Weiterhin ist jede Funktion/Methode mit **mindestens drei Testfällen** im Code zu versehen. Was in diesem Blatt eh Teil der Aufgabenstellung ist und in Form von Doctests umzusetzen ist.
- Achten Sie darauf, dass Ihr Code **lauffähig** ist. Bereiche/Funktionalitäten, die nicht lauffähig sind, werden standardmäßig mit 0 Punkten bewertet.
- Datei- und Ordnernamen sollen keine Umlaute, diakritische oder Sonderzeichen, mit Ausnahme des Unterstrichs, enthalten!
- Ihre Lösung muss ohne Installation von nicht in Python 3.1x enthaltenen Standard-Bibliotheken auskommen. Die Korrektur und Ausführung durch die Tutoren muss also **ohne** ein „pip install ...“ auskommen.

Für dieses Blatt können bis zu **30 Punkte** erreicht werden. Neben der Erfüllung der Aufgabenstellung durch entsprechender Abgabe im Moodle-Kurs, ist ein Code Review bei Ihrer Tutorin/Tutor notwendig. → **kein code review = 0 Punkte**

Im Code Review sollen Sie Ihre Lösung vorstellen. Dies sollte vorbereitet werden und ist Teil der Aufgabe, sprich wird ebenfalls mit bewertet. Bereiche, die im Codereview nicht erklärt werden können, werden ebenfalls mit 0 Punkten bewertet.

Aufgabe

Wir haben einen Graphen, dessen Kanten zwei Werte aufweisen: a) die Kosten/Aufwand, um von einem Knoten zum Anderen zu gelangen und b) einen Ablenkungswert. Für einen solchen Graphen werden ein Start- und ein Ziel-Knoten ausgewählt und es soll der „beste Pfad“ gefunden werden. Dabei sollen die Kosten möglichst gering und der Ablenkungswert möglichst hoch sein.

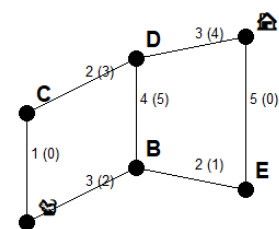
Stellt Euch also vor, es geht um eine Katze 🐈 (im Beispiel-Graphen unten links), die gerade im Garten aufwacht und jetzt Hunger verspürt. Das Essen gibt es im Haus 🏠 (im Beispiel-Graphen oben rechts) und es gibt verschiedene Wege dorthin. Welchen Weg würde die Katze wählen?:

- über B und E zum Haus: Aufwand = $3+2+5 = 10$ und der Ablenkungswert = $2+1+0 = 3$
- über C und D: Aufwand = $1+2+3 = 6$ und der Ablenkwert = $0+3+4 = 7$
- über B und D: Aufwand = $3+4+3 = 10$ und der Ablenkwert = $2+5+4 = 11$
- über C, D, B und E: Aufwand = $1+2+4+2+5 = 14$ und der Ablenkwert = $0+3+5+1+0 = 9$

Würde man erlauben, dass Knoten mehrfach besucht werden können, wären auch Schleife denkbar. Starten wir aber erst einmal ohne. Grundannahme ist also, jeder Knoten darf nur einmal besucht werden.

Variante b) hat den geringsten Aufwand bei trotzdem noch einem Ablenkwert von 7. Dieser Weg wäre daher auf jedem Fall der Variante a) vorzuziehen. Insbesondere, wenn der Hunger groß ist. Variante c) hat jedoch einen höheren Ablenkwert (11).

Wir haben also zwei Parameter, die optimiert werden sollen (man spricht hier von Mehrzieloptimierung, auch Pareto-Optimierung genannt).



Definieren und Beschreiben Eurer Optimierungsfunktionen:

Überlegen Sie verschiedene Varianten, um die Mehrzieloptimierung umzusetzen. Wählen Sie mindestens drei von diesen aus und setzen Sie diese jeweils in eine Funktion um. Überlegen Sie vorab, wie Sie das umsetzen wollen (gibt verschiedene Möglichkeiten)!

Finden eines Weges:

- 1) Implementieren Sie eine Funktion, welche auf dem Greedy-Ansatz basiert. Diese Funktion würde also nicht immer ein globales Optimum finden. Beim Greedy-Ansatz würde die Katze bei dem Beispiel oben, wenn die Mehrzieloptimierung z.B. nur den Ablenkungswert betrachtet, zum B gehen (da $2 > 0$). Anschließend würde die Katze von B zum D gehen (da $5 > 1$) und dann zum Haus. Würde die Verbindung zwischen C und D einen höheren Ablenkungswert als 4 haben, würde unsere Katze vermutlich nicht beim Haus ankommen 😊 ... das wäre bei diesem Aufgabenteil aber erst einmal kein Problem und kann ignoriert werden, aber bitte beachten!
- 2) Implementieren Sie weiterhin eine *rekursive* Funktion, die einen optimalen Weg basierend auf einer der drei Optimierungsfunktionen findet. Bitte beachten, dass es ja auch mehrere Wege geben kann, die gleich gut sind (in diesem Fall muss nur einer dieser angegeben werden). Die Funktion soll sowohl den Aufwand, den Ablenkungswert als auch den entsprechenden Weg *zurückgeben*.

Doctest-Tests:

Erstellen Sie für jede Ihrer Funktionen jeweils mindestens drei Testfälle als Doctest-Tests. Mindestens 1x sollte auch ein Negativ-Test dabei sein!

Machen Sie sich auch klar, dass Testen nicht immer trivial ist. Wenn wir in der rekursiven Funktion 2) verschiedene Lösungsmöglichkeiten haben, muss man hier ein wenig kreativ sein 😊

Zeitmessung:

Machen Sie sich mit den Modulen *time* und *timeit* vertraut. Für die Lösungen zu „Finden eines Weges“ (Greedy 1) und rekursive Funktion 2)) berechnen und vergleichen die benötigte Zeit. Erstellen Sie entsprechende Testdaten (sollten mit Teil der Abgabe sein) und werten Sie Ihre Implementierung entsprechend aus. Der Code dafür soll in ein eigenes Modul ausgelagert werden. Die Ergebnisse können im Kommentar, als CSV-Datei oder auch in einem Tabellenkalkulationsprogramm (wie LibreOffice oder Excel) mitgeliefert werden.

Dokumentation:

Erstellen Sie eine kurze Dokumentation, welche den Tutoren den Einstieg in Ihr Programm und die mitgelieferten .py Dateien erleichtert. Dort sollten also die .py Dateien mit deren Zweck aufgelistet werden (kann Copy-Paste der relevanten DocString Texte sein), sowie deren Zusammenspiel erläutert werden. Sollten Angaben in der Aufgabenstellung fehlen oder nicht eindeutig sein, diese bitte sinnvoll ergänzen: an gegebener Stelle im DocString und in der Dokumentation notieren.

Hinweise:

Eine Eingabe der Graphen wie im Blatt 06 ist nicht nötig. Sie können aber natürlich Ihr Programm von dort anpassen und nutzen, um Testdaten zu erzeugen.

Natürlich ist es auch erlaubt die Situation zu verkomplizieren. Wer also lieber mit gerichteten Graphen arbeiten möchte oder oben erwähnte Zyklen (also das mehrfachbesuche von Knoten) erlauben möchte, darf dies gerne (bitte entsprechend in der Dokumentation angeben).

... werfen Sie auch einen Blick in die Bewertungstabelle! Dort wird genau aufgeführt, wofür Abzüge gemacht werden und was wie viele Punkte gibt. Die Bewertungstabelle ist aber nur eine grobe Richtlinie, die Tutorinnen/Tutoren dürfen davon abweichen.